



sproshchaev

3 июн в 09:20

AI Gateway для микросервисов: гайд по интеграции LLM в 2026

🔗 Сложный

🕒 9 мин

👁 8.5K

Блог компании OTUS, Карьера в IT-индустрии, Java*, Kotlin*, Искусственный интеллект

Тutorial

Всем привет, меня зовут Сергей Прощаев. В этой статье расскажу про одну из самых горячих тем 2026 года — интеграцию AI/ML как самостоятельных сервисов в микросервисную архитектуру. Я Tech Lead и руководитель направления Java/Kotlin разработки в FinTech & E-commerce, также преподаю на курсах разработки и архитектуры в OTUS.

Представьте: вы разрешили сервисам в вашей компании напрямую вызывать ChatGPT. Сначала всё отлично — фичи выкатываются быстро. Но через месяц приходит счёт от OpenAI на \$15K, а вы не понимаете, кто нагенерировал столько токенов. Потом один из сервисов начинает ждать ответа LLM по 3 секунды, и вся продовая цепочка падает. Разработчики добавляют всё новые вызовы, никто не кеширует повторяющиеся вопросы. Знакомо?

За последние полгода я пересмотрел архитектуры четырёх проектов, куда заказчики стремительно добавляли LLM. Картина везде была похожа: сначала все счастливы, потом хаос. Мне как-то попала статистика: до 40% вызовов LLM — повторяющиеся запросы. А ещё один стартап заплатил \$50K за месяц, потому что у них не было кеширования и каждый чих уходил в GPT-4.

К 2026 году AI-модели стали полноценными «гражданами» архитектуры, но подход «дёрни LLM из любого сервиса» больше не работает. Нужен специальный паттерн — **AI Gateway**. В этом гайде я покажу, как спроектировать его как инфраструктурный слой (отдельный сервис, плагин API Gateway или sidecar), добавить семантическое кэширование, контроль стоимости, rate limiting, resilience и безопасность. А главное — как не повторить чужих ошибок.



Рис. 1 — Архитектурный контекст: AI-модели становятся равноправными микросервисами, но требуют специального шлюза

Исходные условия

Прежде чем идти дальше, давайте зафиксируем, с чем мы работаем. Предположим, у вас есть:

- Kubernetes-кластер (версия 1.28+), в котором живут микросервисы.
- Несколько Java/Kotlin-сервисов на Spring Boot. Они уже общаются через REST, возможно, есть API Gateway.
- Доступ к двум-трём LLM: например, OpenAI GPT-4 (для сложных задач) и локальная Llama 3 (для быстрых, дешёвых ответов).
- Трафик к LLM — до 1000 запросов в час, но с пиками.
- Команда из 5–10 разработчиков, никто централизованно не управляет вызовами к моделям.
- Счета за LLM растут на 20–30% месяц к месяцу, и непонятно, почему.

Ограничения на старте: нет готового AI-шлюза, нет кеширования, нет контроля токенов, rate limiting отсутствует. Все вызовы — синхронные и напрямую.

Исходя из этого, мы построим AI Gateway, который решит эти боли.

Что такое AI Gateway и почему Spring Cloud Gateway — хороший вариант

AI Gateway — это выделенный инфраструктурный слой между вашими сервисами и LLM-провайдерами. Он может быть реализован как:

- отдельный микросервис (например, на Spring Cloud Gateway),
- плагин к существующему API Gateway (Kong, APISIX),
- sidecar в mesh-архитектуре (Envoy),
- специализированный inference proxy.

В статье я буду использовать Spring Cloud Gateway для примеров, потому что он реактивный, легко конфигурируется и отлично ложится на Java/Kotlin-стек. Но описанные паттерны универсальны.

AI Gateway управляет:

- маршрутизацией на разные модели (стоимость, SLA, доступность);
- кешированием (прямым и семантическим);
- контролем стоимости (лимиты на сервис/пользователя/сутки);
- rate limiting (запросы/минуту, токены/минуту);
- мониторингом задержек и качества;
- **безопасностью:** фильтрация `prompt injection`, маскирование `PII`, аудит;
- **resilience:** `circuit breaker`, retry policies, bulkhead, защита от `retry storm`.

Пошаговый маршрут: как построить AI Gateway с нуля

Шаг 1. Базовый роутинг

Добавляем зависимость `spring-cloud-starter-gateway`.

В `application.yml` прописываем маршруты. Для демонстрации я использую простейший маршрут по заголовку. В продакшене модель обычно выбирает не клиентский сервис, а policy engine внутри Gateway: на основе SLA, стоимости, типа задачи, текущей доступности моделей и предпочтений пользователя. Клиент просто шлёт запрос

в единый **OpenAI-compatible chat completion API contract** (единый контракт, эмулирующий API OpenAI), а шлюз сам решает, куда его направить.

```
spring:
  cloud:
    gateway:
      routes:
        - id: openai_route
          uri: https://api.openai.com/v1/chat/completions
          predicates:
            - Header=model-type, premium
          filters:
            - name: RequestRateLimiter
              args:
                key-resolver: "#{@userKeyResolver}"
                redis-rate-limiter.replenishRate: 10
                redis-rate-limiter.burstCapacity: 20
        - id: local_llama_route
          uri: http://llama-service:8080/generate
          predicates:
            - Header=model-type, economy
```

Объяснить с  SourceCraft

Проверка: выполните тестовый запрос к Gateway с заголовком `model-type: premium` — он должен уйти на OpenAI. Затем с `model-type: economy` — на локальную Llama. Посмотрите логи Gateway: в них должен быть URI, куда реально пошёл запрос.

Шаг 2. Добавляем кеш результатов

Самый эффективный способ сэкономить. Для точных совпадений (один и тот же промпт) — обычный Redis. Но 80% пользы даст **семантический кеш**: вы храните эмбединги запросов и, если новый запрос семантически близок к сохранённому (например, «Как будет погода завтра в Москве» и «Завтрашний прогноз для Москвы»), возвращаете кешированный ответ.

В production **semantic cache** почти никогда не строится только на **косинусной близости**. Обычно в cache key дополнительно включают:

- tenant/user scope — изоляция данных между разными пользователями;
- model version и system prompt hash;
- domain context.

Кроме того, `cache entries` имеют `TTL` (time-to-live) и должны инвалидироваться при смене `embedding model` или `system prompt version`. Без этого вы рискуете получить устаревшие или неконсистентные ответы. Порог срабатывания (обычно 0,8–0,9) настраивается под конкретную предметную область.

Проверка: сделайте два одинаковых запроса подряд. Первый должен идти в LLM (медленно, например 500 мс). Второй — вернуться из кеша (быстро, 10–30 мс внутри одного региона). Добавьте заголовок `X-Cache: HIT`. Для семантического кеша проверьте два семантически похожих запроса — второй тоже должен быть хитом.

Шаг 3. Контроль стоимости и лимитов

Добавляем фильтр, который считает токены. Каждый запрос логируется в Prometheus. Устанавливаем:

- `requests per minute` — классический `rate limiting`;
- `tokens per minute` — ограничение на объём токенов в минуту;
- `budget per day/per user` — жёсткий финансовый потолок, после которого шлюз возвращает 403.

Для LLM-систем обычного `request rate limiting` недостаточно — токен-ориентированное лимитирование гораздо точнее отражает реальную нагрузку на бюджет.

Полезный трюк: если модель слишком долго отвечает (таймаут), шлюз может переключить запрос на более быструю модель (`fallback`). **Важное уточнение:** `fallback` между моделями безопасен только для `tolerant`-сценариев (чаты, рекомендации, суммаризация). Для `strict-output` задач (JSON extraction, agents с tool calling) `fallback` может нарушить контракт ответа. В таких случаях лучше вернуть ошибку.

Проверка: превысьте лимит токенов — получите 429. Для `fallback`: отключите основную модель в тестовой среде — проверьте переключение на резервную.

Шаг 4. Мониторинг и observability

Каждый вызов LLM должен оставлять трейс (OpenTelemetry). Метрики:

- `latency p50/p95/p99` на модель;
- стоимость вызова;
- кеш-хиты;
- частота `rate limiting`;
- количество срабатываний `circuit breaker`.

Для resilience-слоя желательно добавить адаптивное поведение — например, автоматическую настройку порогов `circuit breaker` на основе наблюдаемых ошибок и задержек (`auto tuning`). Это уже следующий уровень зрелости, но его полезно иметь в виду.

Проверка: выполните несколько запросов, зайдите в Jaeger/Grafana. Должны быть спаны `security-filter`, `cache-check`, `llm-call`, `resilience`.

Шаг 5. Streaming-ответы

`Streaming` (SSE/WebFlux) требует отдельного внимания. Gateway должен проксировать поток без буферизации (иначе теряется смысл стриминга) и **по возможности отменять запрос к upstream-модели** при `client disconnect`. Важно: поддержка `cancellation` зависит от конкретного LLM-провайдера — некоторые API продолжают `inference` даже после разрыва соединения и биллят полную генерацию. **Семантика биллинга (`billing semantics`) при частичной генерации также зависит от провайдера: могут списать токены за весь сгенерированный ответ, даже если клиент отключился.** Это нужно учитывать при выборе провайдера и проектировании.

Проверка: начните генерацию длинного ответа через Gateway и закройте вкладку браузера. В логах должно быть событие отмены (если провайдер поддерживает). Убедитесь, что потребление токенов прекратилось.

Шаг 6. Resilience: circuit breaker, bulkhead, retry storm

AI Gateway быстро становится критической инфраструктурой. LLM-провайдеры могут деградировать, выдавать 5xx, `rate limit`ить или зависать на десятки секунд. Без защиты один проблемный провайдер положит все сервисы.

Что добавляем:

- **Circuit Breaker** — при достижении порога ошибок (например, 50% ошибок за 10 секунд) шлюз перестает отправлять запросы к этому провайдеру и быстро возвращает `fallback`-ответ или ошибку. **Желательно настроить автоматическую подстройку порогов на основе наблюдаемых метрик (`adaptive circuit breaker`).**
- **Bulkhead isolation** — пул потоков/соединений для каждого провайдера, чтобы медленный провайдер не забирал все ресурсы.
- **Защита от `retry storm`** — ограничение количества повторных попыток (обычно 1–2) с экспоненциальной задержкой, иначе при сбое провайдера все сервисы начнут ретраить одновременно и добьют остатки.

В Spring Cloud Gateway это реализуется через `Resilience4J` фильтры. Пример конфигурации `circuit breaker`:

```
filters:
  - name: CircuitBreaker
    args:
      name: openaiCB
      fallbackUri: forward:/fallback/openai
```

Объяснить с 

Проверка: в тестовой среде заставьте провайдера возвращать 500 ошибку. Circuit breaker должен разомкнуться после нескольких запросов, и последующие запросы сразу идти в fallback (не доходя до провайдера). Через заданное время (например, 10 секунд) перейти в half-open и проверить восстановление.

Визуализируем диалог: как работает AI Gateway с кешем

Прежде чем показывать схему, представьте: клиентский сервис (например, «рекомендации») хочет спросить у LLM: «Какие товары подойдут к красному пальто?». Вместо прямого вызова OpenAI он идёт к AI Gateway.

Вот как Gateway обрабатывает запрос — проверяет кеш, при промахе идёт к LLM, сохраняет результат и отдаёт клиенту (рис. 2).

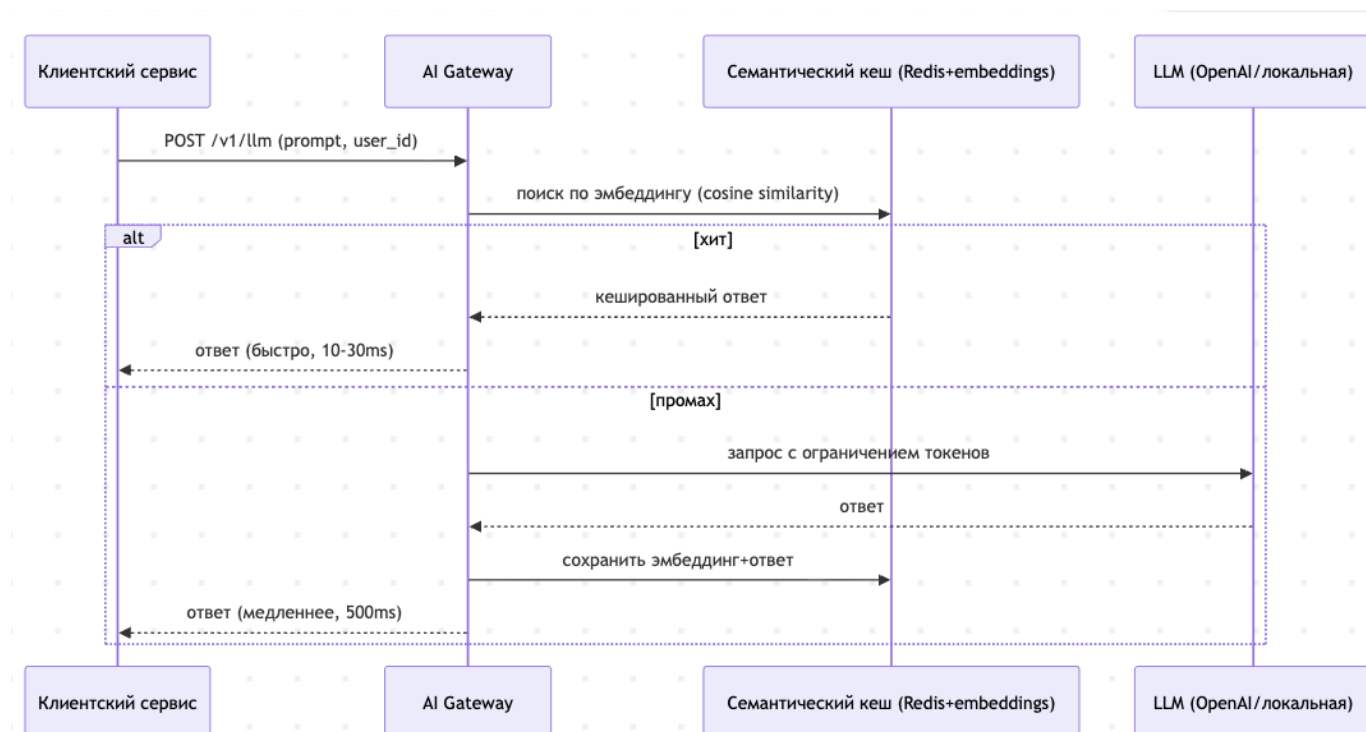


Рис. 2 — Sequence diagram: как AI Gateway сначала проверяет семантический кеш и только при промахе идёт к LLM

Из диаграммы видно, что большая часть повторяющихся или похожих запросов никогда не долетает до платной LLM. Время ответа при cache hit — 10–30 мс, что на порядок

быстрее вызова LLM. Кеш работает на уровне смыслов, а не точного совпадения, удерживая счета под контролем.

Внутреннее устройство AI Gateway: компоненты

Заглянем внутрь AI Gateway и посмотрим как запрос проходит через фильтр безопасности, аутентификацию, rate limiter, resilience, роутер на модель, кеш и сбор метрик (рис. 3).

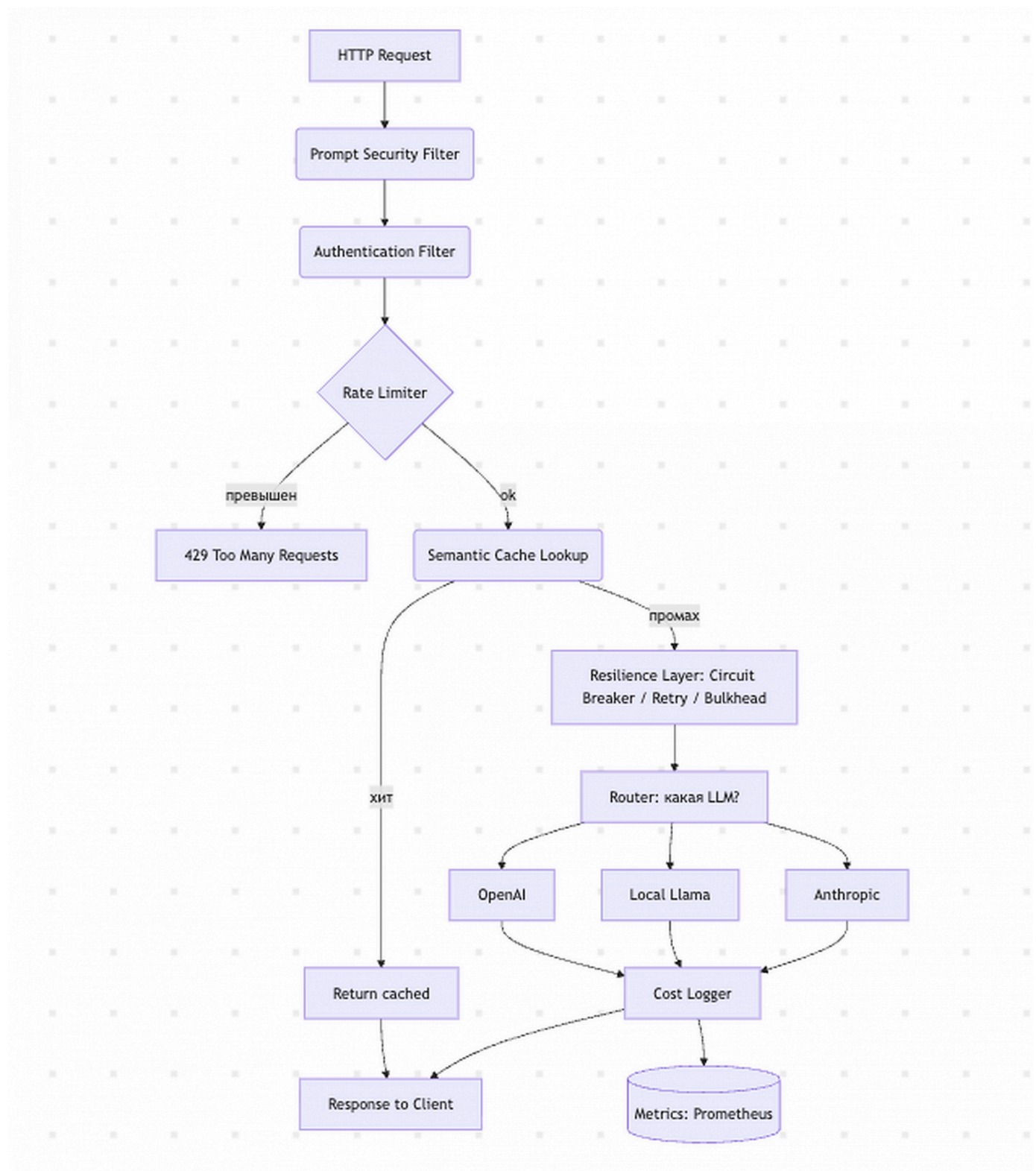


Рис. 3 — Component diagram: из каких частей состоит AI Gateway, включая resilience

Вы можете собрать такой шлюз из стандартных строительных блоков. В enterprise-сценариях Gateway становится точкой security-контроля (Prompt Security Filter проверяет jailbreak-паттерны, маскирует PII, ведёт аудит). Resilience Layer защищает систему от деградации одного провайдера. Аутентификация, rate limiting (включая token-based), кеш, роутинг — всё это кастомные фильтры или готовые реализации. В 2026 году многие компании уже выложили свои наработки в open source, например Envoy AI Gateway.

Где это решение не сработает (ограничения)

1. **У вас всего один сервис и 100 запросов в день.** Просто встройте вызов LLM в этот сервис — шлюз добавит сложности.
2. **Все запросы уникальны и имеют низкую семантическую повторяемость.** Кеш не поможет, но rate limiting и контроль стоимости всё равно полезны.
3. **Streaming с очень жёсткими требованиями к задержке.** Шлюз добавляет микро-задержку.
4. **Нет возможности поддерживать ещё один компонент.** AI Gateway — это отдельный слой, его нужно деплоить, мониторить, обновлять.

В остальных 80% случаев — да, стоит строить.

Реальный кейс: как Uber построил AI Gateway

Мне попался отличный пример из публичных источников — опыт Uber. В 2024–2025 годах они столкнулись с тем, что более 100 микросервисов начали напрямую вызывать LLM. Счета росли, разработчики жаловались на непредсказуемые задержки.

Uber разработал внутренний AI Gateway с тремя ключевыми особенностями:

1. **Автоматическая маршрутизация на основе стоимости и SLA.** Запрос к рекомендациям шёл на дешёвую модель, критичный диалог поддержки — на самую умную.
2. **Динамический выбор модели** в зависимости от загрузки кластера. Если локальная LLM была свободна — использовали её, если перегружена — переключались на OpenAI.
3. **Семантический кеш с TTL, зависящим от домена.** Для поиска товаров кеш жил 15 минут, для персональных рекомендаций — 1 минуту.

Результат: централизация доступа, единые политики безопасности, кеширование и fallback. Согласно публичным данным, GenAI Gateway обрабатывает более 5 миллионов запросов в день, обслуживая свыше 40 внутренних сервисов, и предоставляет единый

OpenAI-compatible chat completion API contract к моделям от OpenAI, Vertex AI и внутренним LLM.

Я бы предпочёл такой подход простой «ковровой бомбардировке» вызовов. Он системный и масштабируемый. В крупных системах подобная централизация часто окупается очень быстро — особенно при росте количества команд и LLM-интеграций.

Чек-лист перед внедрением

- У нас больше двух сервисов, вызывающих LLM?
- Месячный счёт за модели превышает \$1000?
- Бывают ситуации, когда один сервис случайно забивает лимиты для всех?
- Разработчики жалуются на непредсказуемые задержки?
- Мы хотим попробовать разные LLM без переписывания кода?

Если ответили «да» хотя бы на два пункта — AI Gateway вам нужен!

Подведём итог

AI-модели перестали быть экзотикой. В 2026 году они — обычные микросервисы, но с платным доступом и переменной задержкой. AI Gateway (инфраструктурный слой, а не просто ещё один сервис) позволяет управлять ими централизованно: экономить бюджет, обеспечивать безопасность (prompt injection, PII, аудит), добавлять resilience (circuit breaker, bulkhead) и не падать в прод при сбоях провайдеров.

Главный вывод: не ждите, пока хаос начнёт гореть. Стройте шлюз заранее. Он окупится быстрее, чем вы напишете очередной прямой вызов OpenAI из нового сервиса.

Что дальше?

В этом гайде я показал архитектуру и ключевые компоненты. Если хотите не просто прочитать, а научиться собирать такой шлюз руками на реальных проектах — обратите внимание на [курс](#) OTUS «**Архитектура и шаблоны проектирования**».

А перед стартом курса можно прийти на бесплатный открытый урок:

- **15 июня в 20:00** «Системы обмена сообщениями: RabbitMQ и Kafka». Записаться
- Разберем два популярных брокера сообщений — RabbitMQ и Kafka: их архитектурные особенности, основные принципы работы и сценарии, в которых один инструмент подходит лучше другого. Это хороший способ познакомиться с преподавателем, протестировать формат обучения и задать вопросы по теме.

Если хотите выбрать не один вебинар, а сразу посмотреть, какие темы будут полезны именно вам, полный список бесплатных уроков июня смотрите в дайджесте.

Теги: java, AI Gateway, LLM, Spring Cloud Gateway, semantic cache, circuit breaker, microservices architecture, OpenAI API

Хабы: Блог компании OTUS, Карьера в IT-индустрии, Java, Kotlin, Искусственный интеллект



Рассылка Хабра для бизнеса

О том, как пиарщикам, маркетологам и эйчарам взаимодействовать с IT-аудиторией

Электронная почта

Подписаться

Дайджест может содержать маркетинговые материалы. Подписываясь, вы соглашаетесь на их получение и обработку данных согласно Политике конфиденциальности



256K+

Охват за 30 дней

OTUS

[Сайт](#) [ВКонтакте](#) [Telegram](#)



64K+

22

Охват за 30 дней

Карма

Sergey Proshchaev @sproshchaev

Tech Lead | Java | Kotlin developer

Подписаться



РЕКЛАМА



Что у банка под капотом?



Узнай в блоге ПСБ



Комментарии



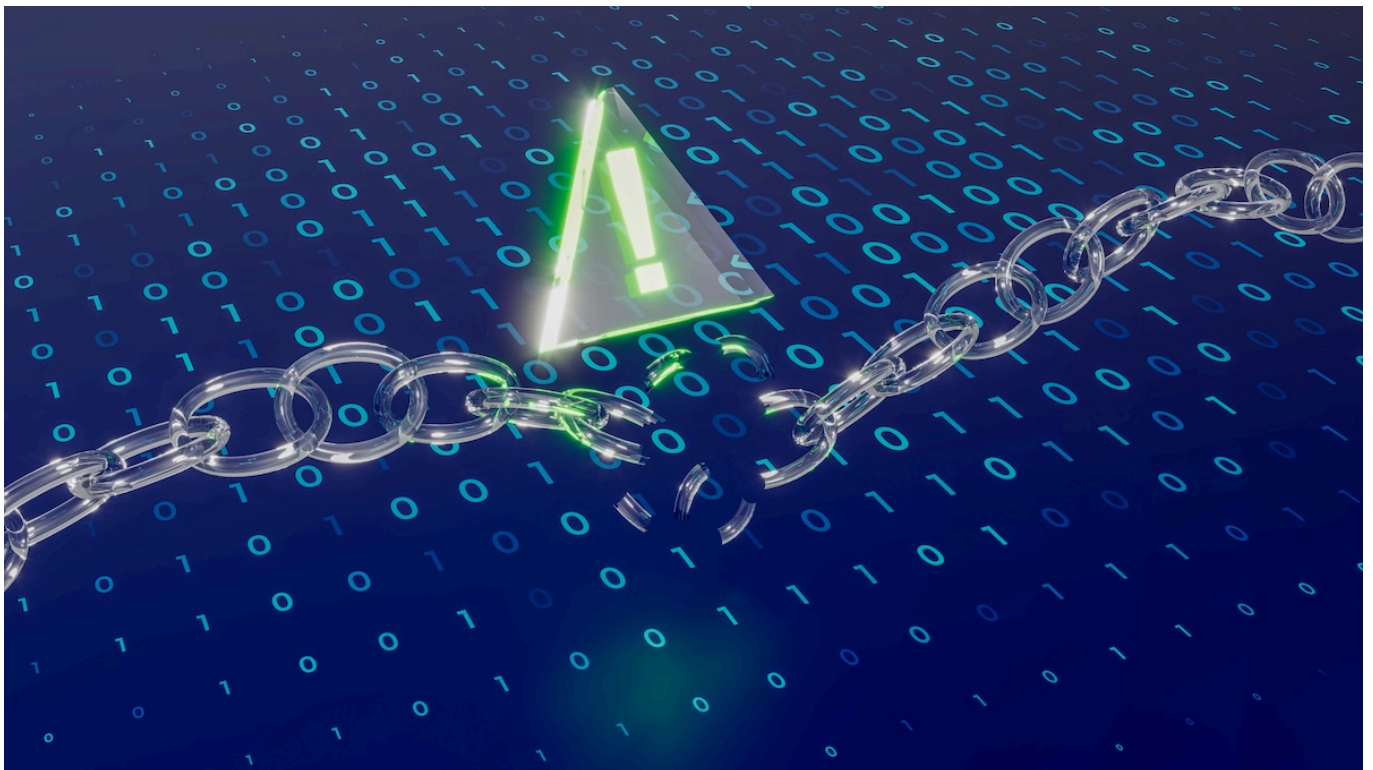
Здесь пока нет ни одного комментария, вы можете стать первым!

Зарегистрируйтесь на Хабре, чтобы оставить комментарий

ЕЖЕДНЕВНЫЙ ХАБР | 29 СЕНТ 2026



70K



Криптографы показали практическую подделку RSA-подписей без факторизации ключа

Новое исследование представило метод, использующий классические вычислительные мощности для снижения текущего уровня безопасности RSA до неприемлемо низкого предела. Применение атаки к устаревшим 1024-битным ключам заняло всего несколько месяцев на академическом...

Как зарабатывать на Open Source без смены лицензии: разбор Cap (22.8k звёзд) и схема с платными бинарниками

Семь привычек Linux-инженера, которые спасают прод

Что такое OpenSpec? Полный гайд по Spec-Driven Development

Ещё



Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



melnik909
9 часов назад

HTML и CSS помогут вам сделать ужасный интерфейс для пользователя

Средний 6 мин 7K

Обзор

 +49

 18

 7



ToxaBes

21 час назад

Путь к ИИ-сингулярности



Простой



16 мин



12K

Мнение

 +43

 30

 24



dalerank

21 час назад

Coyote Time или как немножко врать игрокам



Простой



8 мин



11K

Кейс

 +34

 14

 7



prost_arina

10 часов назад

От чата с LLM к полноценному AI-агенту: пошаговая настройка OpenCode



9 мин



6.9K

 +32

 45

 0



Suor

18 часов назад

[Не]нужные люди в эпоху ИИ



7 мин



12K

 +27

 15

 55



vladbalv

11 часов назад

Кто отвечает за ИИ-агента: юридический контур цифрового сотрудника



16 мин



5.5K

 +24

 6

 1



proton17
7 часов назад

«Железо» под давлением: как протестировать тысячи плат для базовых станций

 17 мин  5.7K

 +23

 9

 4



PatientZero
7 часов назад

Пишем трассировщик лучей на Brainfuck

 9 мин  5.9K

Перевод

 +21

 4

 1



grigoriy-melnikov
11 часов назад

Как построить сервис с нуля самому. ПО ШАГАМ

 Простой  10 мин  5.4K

Мнение

 +21

 12

 5



medicalofficer1111
11 часов назад

Почему нельзя просто залить бактерии на лице антисептиком

 8 мин  4.7K

 +19

 14

 0

Как построить архитектуру облака под проект: что учесть и где подводные камни

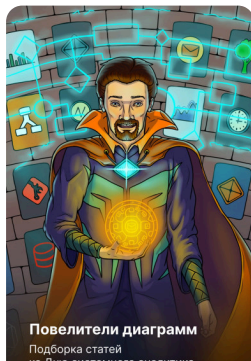
Турбо

Показать еще

ИСТОРИИ



Тебе бы в ИБ, а ты
всё на драккаре
плаваешь



Все ответы у них



Проблемы?
Внимательно вас
слушаю



Годнота из блогов
компаний

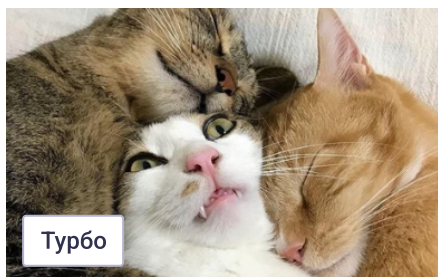


Маркетологи,
срочно в санаторий!

МИНУТОЧКУ ВНИМАНИЯ



Как работает Kafka, где брокеры
не владеют данными



Когда одному VPS уже тесно.
Строим архитектуру облака



ИИ ускорил разработчиков, но
не команды: что показал опрос



Настройка языка

